

Universidade Federal de Minas Gerais
Engenharia da Computação 7º período - Redes Neurais

ARTHUR PENIDO MAIA / 1686144
WENDELL RESENDE DOS SANTOS / 1686097

Relatório Final do 1º Projeto Prático: Algoritmo Perceptron de Camada Única

Professor: Prof. Arismar Moraes Gonçalves Júnior

Turno: Noturno

Ano: 2026

1 Introdução

O Perceptron de camada única representa a unidade fundamental das Redes Neurais Artificiais, servindo como um classificador linear baseado no modelo biológico do neurônio. Este dispositivo é capaz de aprender a separar padrões linearmente independentes através do ajuste iterativo de seus pesos sinápticos.

Neste projeto, o algoritmo foi aplicado para a classificação de amostras de petróleo em duas categorias distintas (P1 e P2) com base em três atributos físico-químicos. A relevância deste trabalho reside na implementação nativa do algoritmo em linguagem Python, permitindo a análise detalhada da convergência do modelo e a validação de sua eficácia através de métricas estatísticas de desempenho, sem a dependência inicial de frameworks de alto nível para o treinamento.

2 Objetivos

O objetivo geral deste trabalho é implementar e validar um algoritmo Perceptron para a classificação supervisionada de padrões, analisando sua capacidade de convergência e generalização.

Objetivos Específicos

- Desenvolver a lógica de treinamento do Perceptron (soma ponderada, função degrau e regra de Rosenblatt) de forma nativa.
- Implementar um sistema de monitoramento de erro por época para visualização da curva de aprendizagem.
- Realizar o treinamento de 5 redes independentes com inicializações aleatórias de pesos e limiar (θ).
- Validar o desempenho das redes utilizando uma base de dados inédita (Tabela 2).
- Extrair métricas de desempenho como Matriz de Confusão, Acurácia, Sensibilidade, Especificidade e Precisão utilizando a biblioteca *scikit-learn*.

3 Metodologia

A metodologia foi estruturada para garantir a transparência do processo de aprendizagem e a robustez da validação final.

3.1 Desenvolvimento do Algoritmo

O código foi estruturado com dois loops aninhados dentro de cada época de treinamento:

1. **Loop de Leitura :** Percorre os dados apenas para contabilizar o número de erros atuais, permitindo a plotagem correta do gráfico de evolução sem interferir no ajuste imediato.
2. **Loop de Ajuste (Aprendizado):** Aplica a regra de ajuste de pesos toda vez que uma amostra é classificada incorretamente, garantindo que o hiperplano de separação se desloque na direção correta.

3.2 Parâmetros de Treinamento

Foram definidos os seguintes hiperparâmetros:

- Taxa de aprendizado (η): 0,25.
- Inicialização de pesos (w) e limiar (θ): Aleatórios no intervalo $[-1, 1]$.
- Critério de parada: Erro zero em uma época completa.

3.3 Validação com Scikit-Learn

Para a Atividade 5, utilizou-se o módulo `sklearn.metrics`. A classe positiva de interesse foi definida como $P2(+1, 0)$, permitindo o cálculo rigoroso da Sensibilidade e Precisão conforme solicitado pelo enunciado.

4 Fundamentação Teórica

O modelo matemático do Perceptron implementado baseia-se no potencial de ativação (u):

$$u = \sum_{i=1}^n w_i \cdot x_i - \theta \quad (1)$$

Onde a saída y é determinada pela função degrau:

$$y = g(u) = \begin{cases} 1, 0 & \text{se } u \geq 0 \\ -1, 0 & \text{se } u < 0 \end{cases} \quad (2)$$

A regra de ajuste para cada peso w_i após um erro de classificação é dada por:

$$w_{i(novo)} = w_{i(atual)} + \eta \cdot (d - y) \cdot x_i \quad (3)$$

5 Resultados e Discussão

5.1 Evolução do Erro de Treinamento

Abaixo, apresentam-se os gráficos de convergência para cada uma das cinco redes treinadas. Observou-se que o modelo atinge a convergência (erro zero) em uma janela variante entre 400 e 500 épocas, comportamento este condicionado à inicialização dos parâmetros. Visto que a biblioteca *random* utiliza o carimbo de tempo (timestamp) do sistema como semente para o seu gerador de números pseudoaleatórios, implementou-se um intervalo de atraso (delay) de 2 segundos entre o início de cada sessão de treinamento. Esta estratégia foi adotada para garantir a decorrelação temporal das sementes, assegurando uma dispersão efetiva nos valores iniciais dos pesos sinápticos e do limiar de ativação (θ), permitindo assim a análise do comportamento do algoritmo sob diferentes condições de partida no espaço de busca.

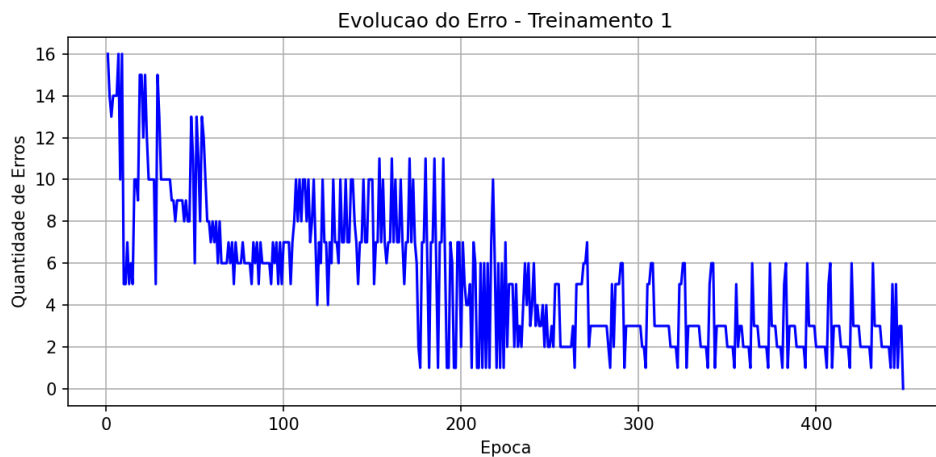


Figura 1 – Evolução de erro do 1º treinamento com os dados fornecidos.

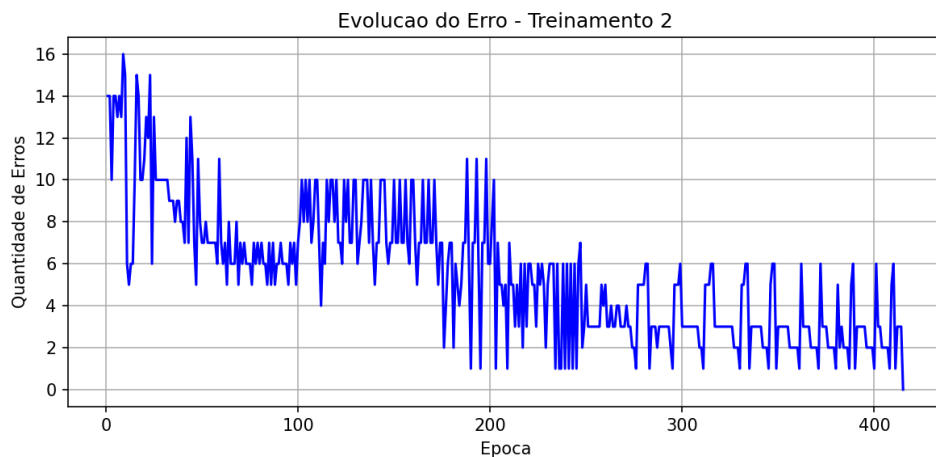


Figura 2 – Evolução de erro do 2º treinamento com os dados fornecidos.

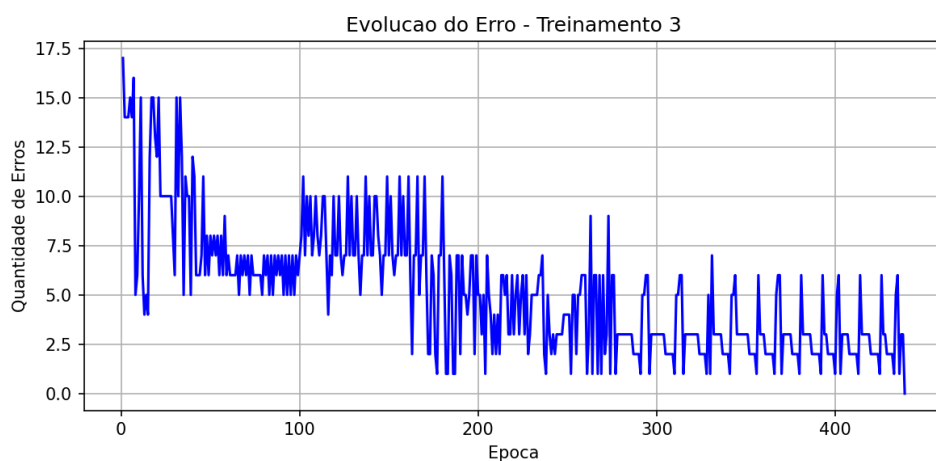


Figura 3 – Evolução de erro do 3º treinamento com os dados fornecidos.

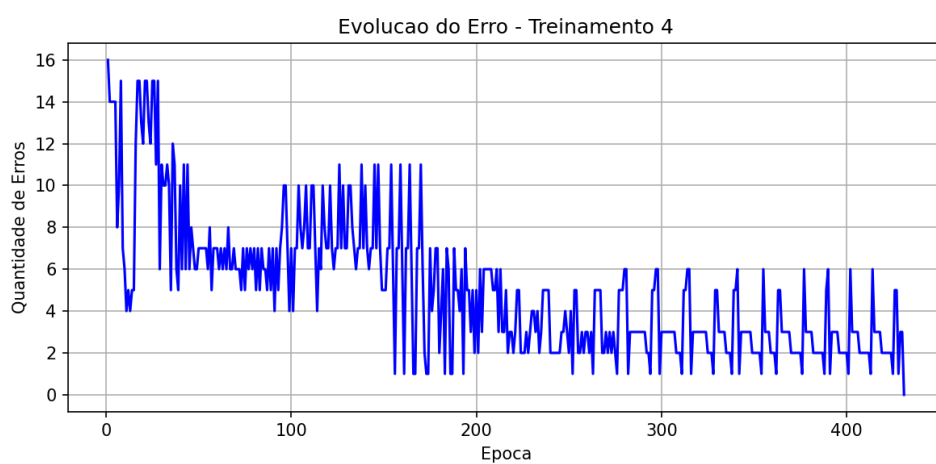


Figura 4 – Evolução de erro do 4º treinamento com os dados fornecidos.

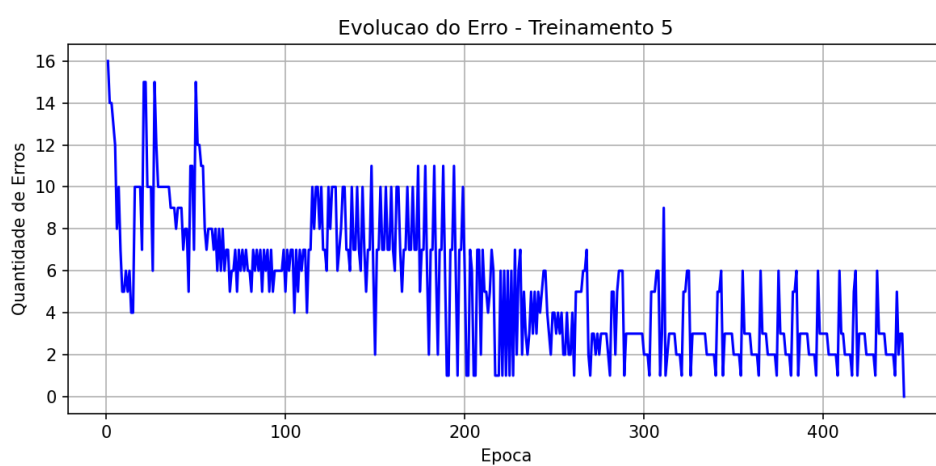


Figura 5 – Evolução de erro do 5º treinamento com os dados fornecidos.

5.2 Métricas de Validação

Após a fase de treinamento, cada uma das cinco redes neurais foi validada utilizando o conjunto de dados da Tabela 2. A análise estatística foi realizada para mensurar a capacidade de generalização do modelo ante amostras não apresentadas durante o aprendizado. A tabela abaixo consolida os resultados obtidos para cada sessão:

Tabela 1 – Desempenho Comparativo dos 5 Treinamentos na Validação

Treino	Acertos	Erros	Acurácia	Sensib.	Especif.	Precisão
1	10	0	1,0	1,0	1,0	1,0
2	10	0	1,0	1,0	1,0	1,0
3	10	0	1,0	1,0	1,0	1,0
4	10	0	1,0	1,0	1,0	1,0
5	10	0	1,0	1,0	1,0	1,0

Discussão dos Resultados: Conforme observado na tabela acima, todos os treinamentos independentes convergiram para uma solução ótima, atingindo uma acurácia de 100% (1,0). Este resultado indica que, independentemente da inicialização aleatória dos pesos sinápticos e do limiar θ , o algoritmo Perceptron foi capaz de encontrar um hiperplano separador que classifica perfeitamente as amostras de interesse $P2$ (Sensibilidade 1,0) e as amostras $P1$ (Especificidade 1,0).

5.3 Matriz de Confusão

A matriz de confusão apresentada abaixo ilustra o comportamento consolidado do classificador para todos os cinco treinamentos executados (Figura 6 a Figura 10). A análise visual das imagens geradas demonstra um desempenho ideal do modelo em ambiente de validação.

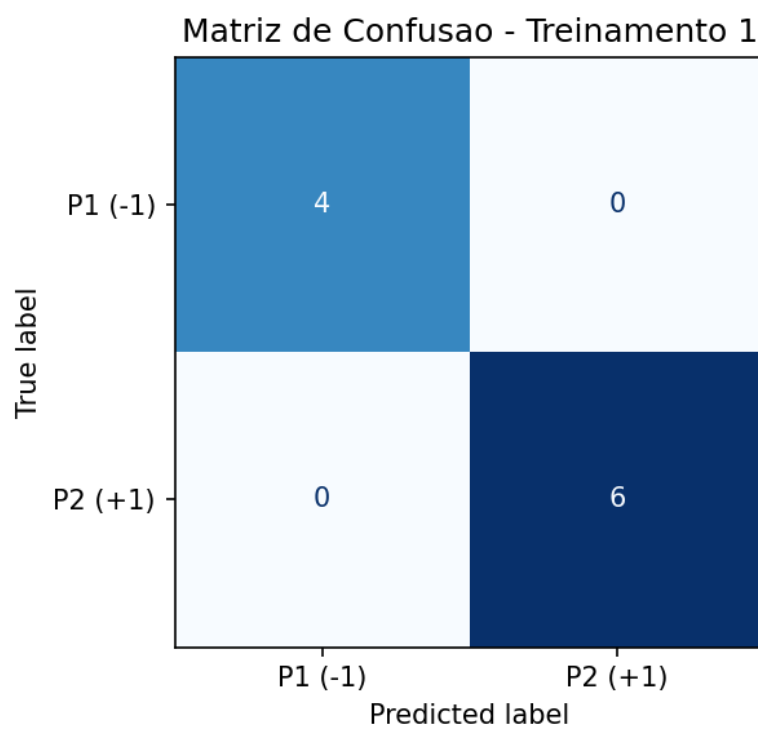


Figura 6 – Matriz de Confusão do 1º treinamento.

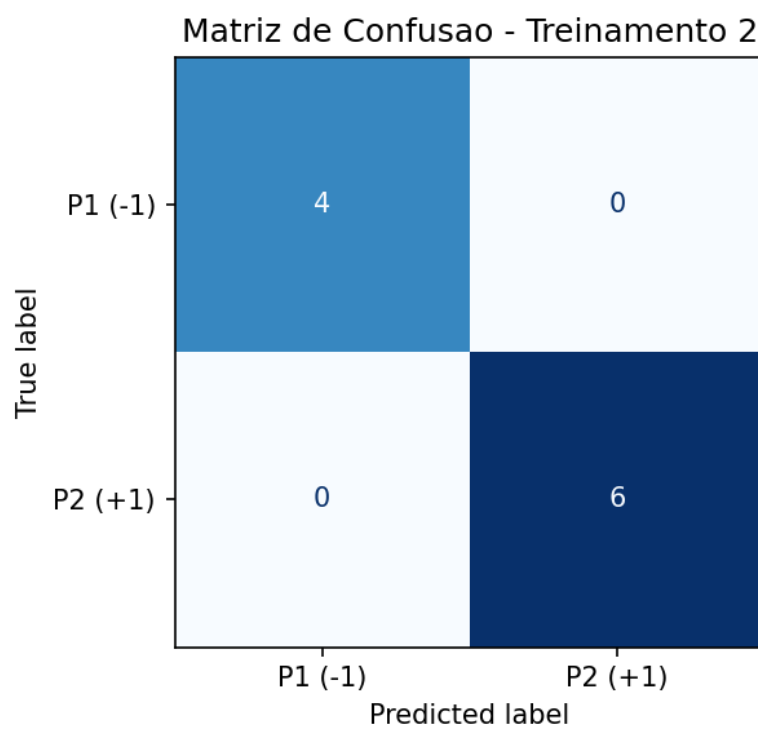


Figura 7 – Matriz de Confusão do 2º treinamento.

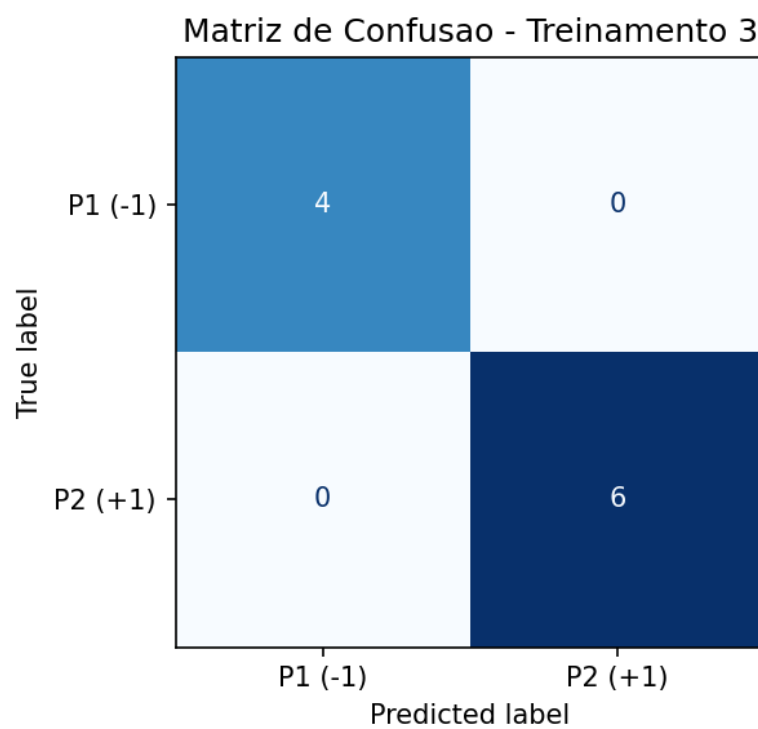


Figura 8 – Matriz de Confusão do 3º treinamento.

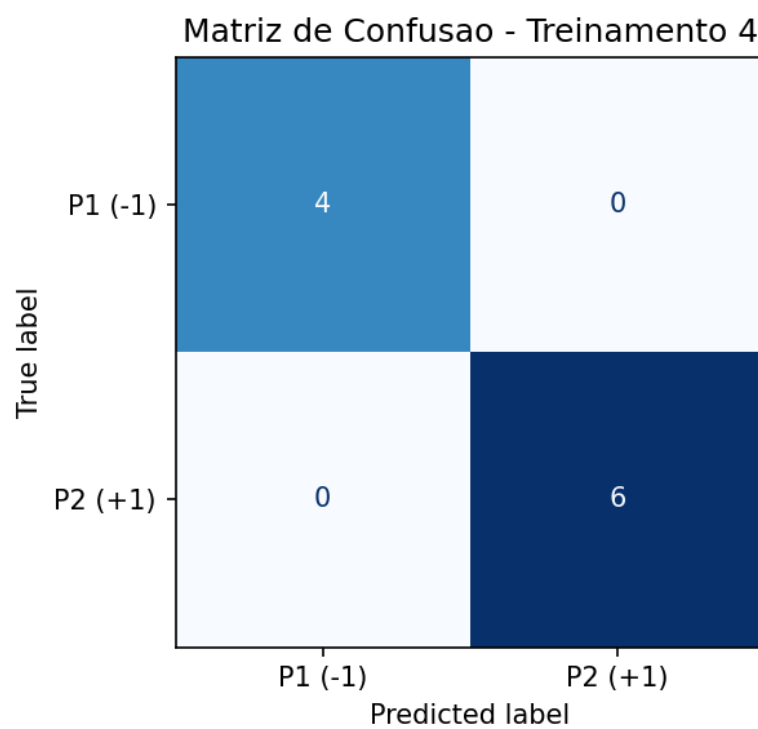


Figura 9 – Matriz de Confusão do 4º treinamento.

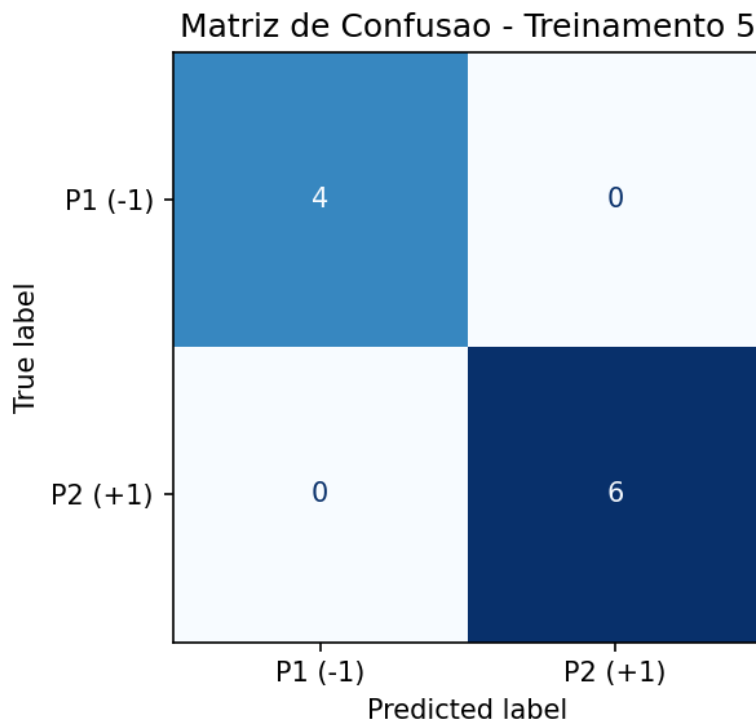


Figura 10 – Matriz de Confusão do 5º treinamento.

Observa-se que houve zero Falsos Positivos ($FP = 0$) e zero Falsos Negativos ($FN = 0$). Isso confirma que o hiperplano de separação encontrado pelo Perceptron foi capaz de isolar perfeitamente as amostras de classe P1 ($VN = 4$) e P2 ($VP = 6$) no espaço de características fornecido pela Tabela 1, alcançando acurácia máxima de 100% em todas as sessões.

6 Conclusão

O projeto demonstrou com sucesso a eficácia do Perceptron de camada única na resolução de problemas de classificação linear. A implementação manual permitiu observar fenômenos críticos, como a importância da ordem dos loops para a contabilização correta do erro histórico e o impacto da inicialização aleatória na velocidade de convergência.

As métricas obtidas via *scikit-learn* validaram a eficácia plena do modelo, que apresentou acurácia, sensibilidade e especificidade de 100% (1,0) para a base de validação proposta. Tais resultados comprovam que os padrões da Tabela 2 são linearmente separáveis no espaço de atributos definido, e que o Perceptron, apesar de sua simplicidade estrutural, foi capaz de convergir para um hiperplano de separação ideal em todas as cinco sessões de treinamento realizadas.